

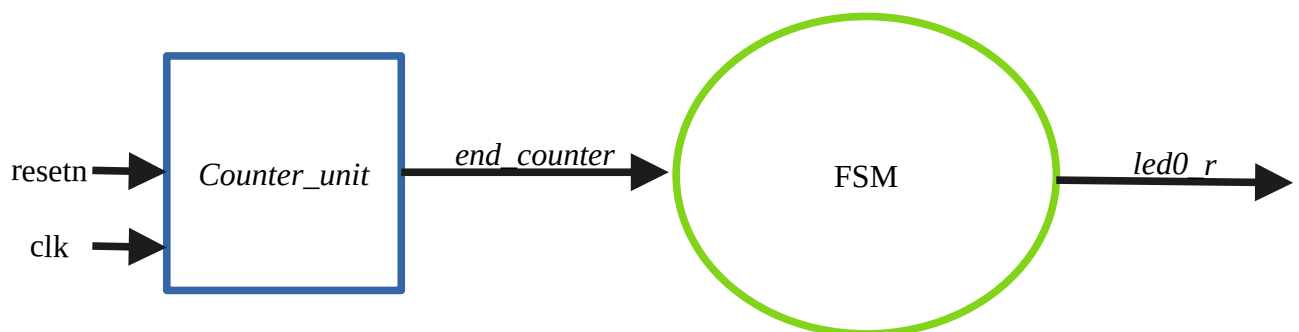
TP4

Pilotage de LED et mémoire

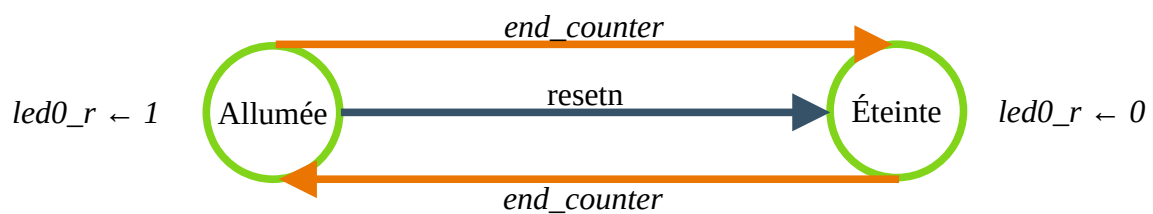
Partie I

1)

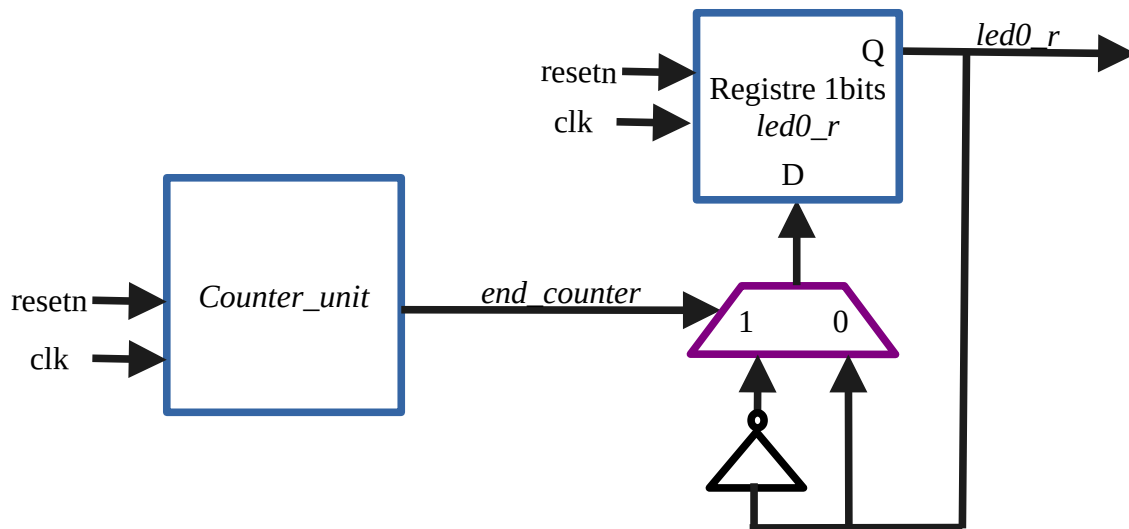
Voici le schémas RTL d'une structure permettant de faire clignoter une led rouge avec une machine d'état.



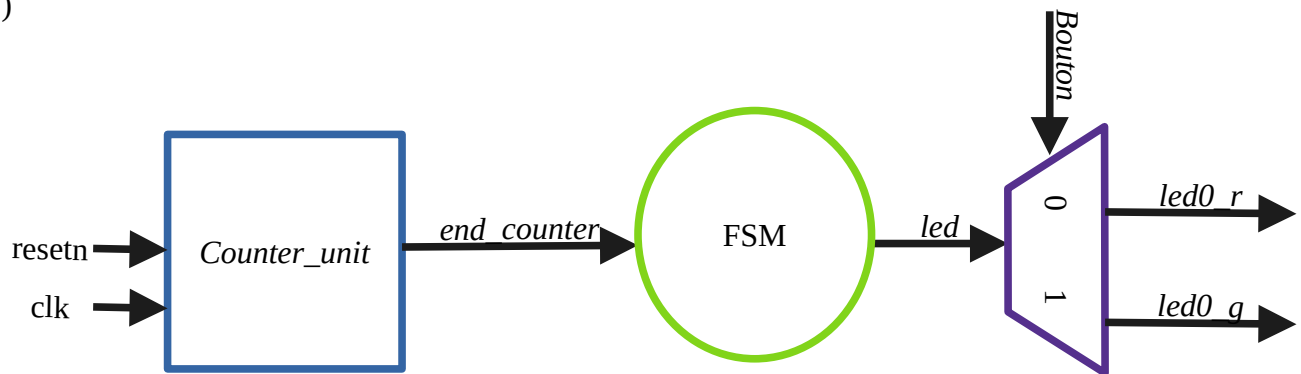
La machine d'état à seulement deux états : « Allumée » et « Éteinte ».



Cette machine d'état peut aussi être simplement remplacée par un registre et un multiplexeur.
led0_r change d'état lorsque *end_counter* est à 1.



2)



La machine d'état ne pilote que le signal *led* qui allume ou éteint une led.
 Le bouton sélectionne la led à faire clignoter. Cette partie peut être réalisée de manière combinatoire.

3)

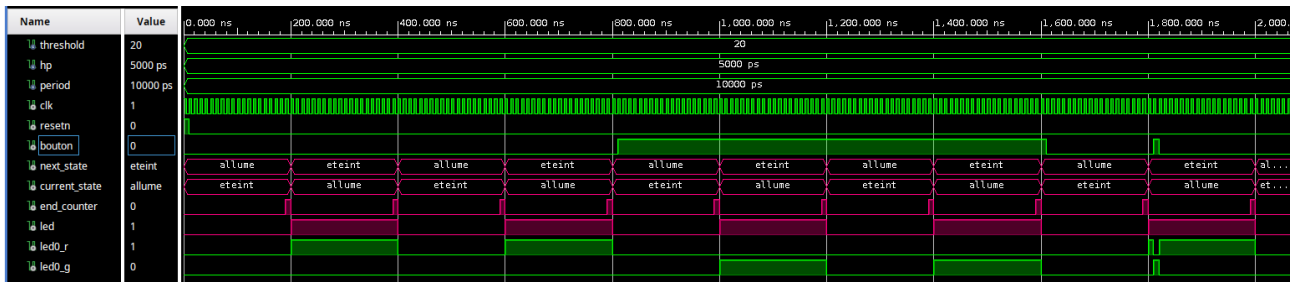
Voici le process de la machine d'état.

```
process(current_state)
begin
  case current_state is
    when eteint =>
      next_state <= allume;
      led <= '0';
    when allume =>
      next_state <= eteint;
      led <= '1';
  end case;
end process;

led0_r <= led when (bouton = '0') else '0';
led0_g <= led when (bouton = '1') else '0';
```

4)

Le testbench de cette architecture est écrit dans le fichier *tb_led_pilot_q4.vhd*.



En rose les signaux internes à *led_pilot*.

Après le reset, tous les signaux sont à 0, *current_state* est à « éteint » et *next_state* à « allumé ».

Lorsque le compteur a terminé il envoie l'information par *end_counter*.

Alors, *current_state* passe à « allumé » et *next_state* à éteint.

Le signal *led* passe alors à 1 et déclenche l'allumage d'une des 2 led.

Le bouton n'étant pas appuyé, c'est la led rouge qui s'allume. Le peut le voir car le signal *led0_r* est à 1.

Après 2 clignotement de la led rouge, on déclenche l'appui sur le bouton. On peut voir que c'est maintenant la led verte qui s'allume car le signal *led0_g* passe à 1.

On relâche le bouton afin de tester son comportement lorsqu'il n'est actif que pendant un cycle d'horloge.

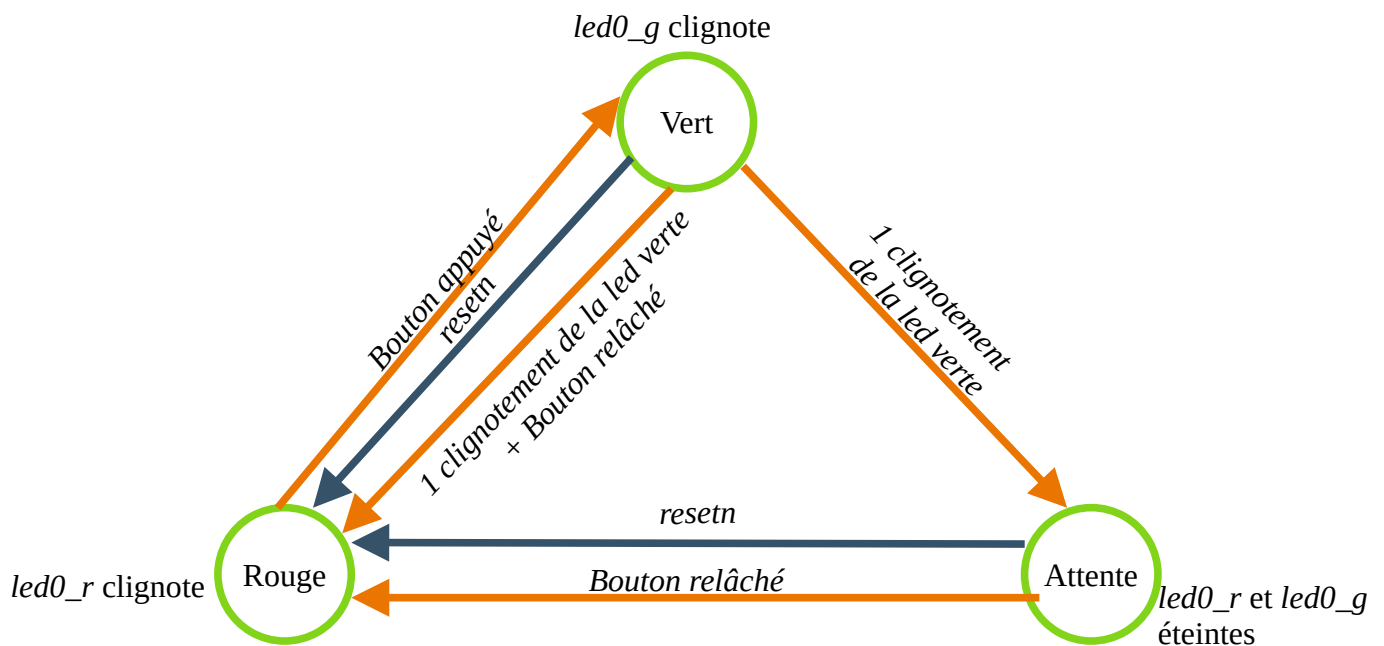
La led verte s'allume uniquement pendant le bref appui du bouton. La led rouge reprend le relais dès que le bouton est relâché.

5)

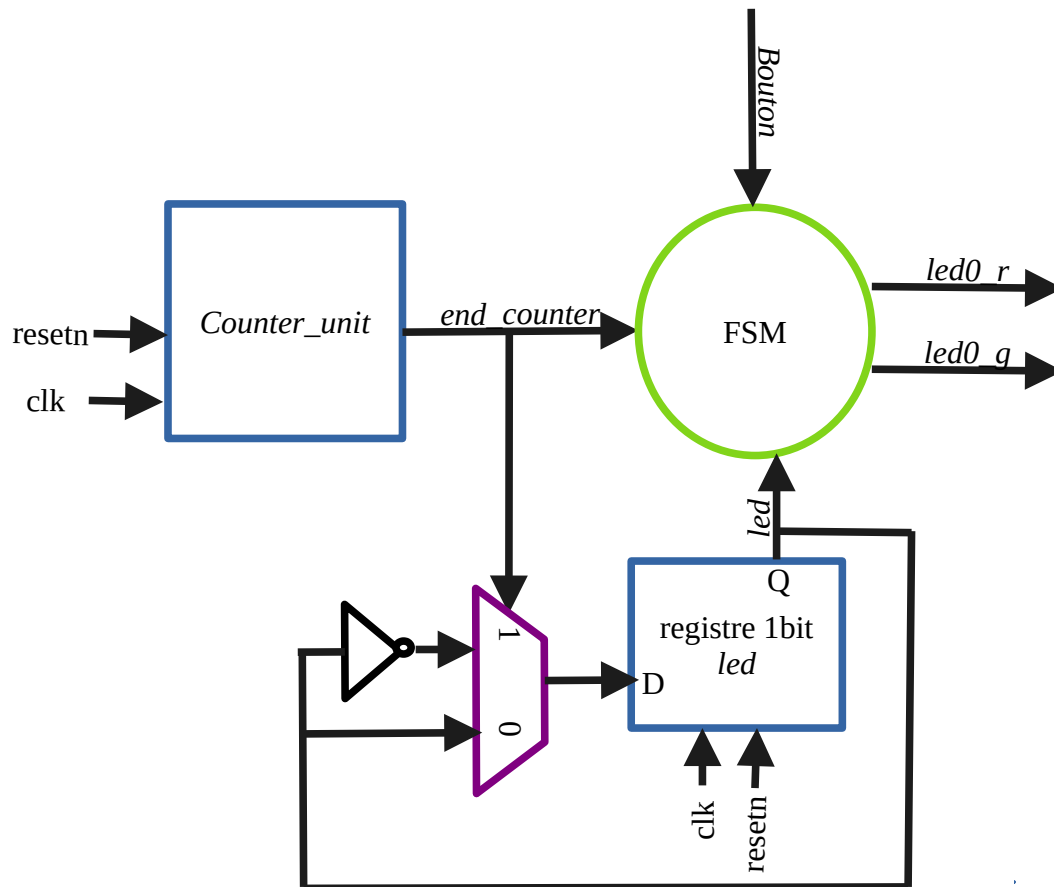
Pour ne faire clignoter la led verte qu'une seule fois, on peut changer la machine d'état.

Au lieu de 2 états allumé et éteint, il y a 3 états : Rouge, Vert et Attente.

Cette machine d'état ne gère plus le clignotement mais la led à faire clignoter.



6)



7)

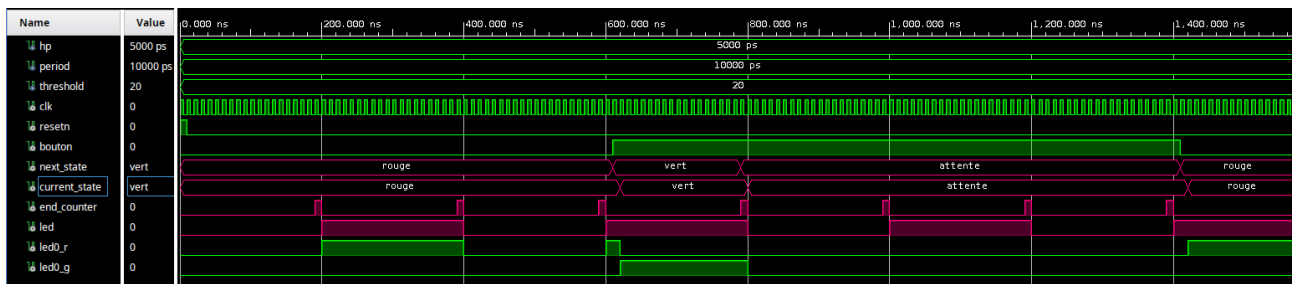
Voici le process de la machine d'état.

```
process(current_state, end_counter, bouton)
begin
  case current_state is
    when rouge =>
      led0_r <= led;
      led0_g <= '0';
      if( bouton = '1') then
        next_state <= vert;
      else
        next_state <= rouge;
      end if;

    when vert =>
      led0_r <= '0';
      led0_g <= led;
      if(end_counter = '1' and led = '1' and bouton = '1') then -- appui long -> on passe en attente
        next_state <= attente;
      elsif(end_counter = '1' and led = '1' and bouton = '0') then -- appui court -> on passe directement à rouge
        next_state <= rouge;
      else next_state <= vert; end if;

    when attente =>
      led0_r <= '0';
      led0_g <= '0';
      if( bouton = '0') then
        next_state <= rouge;
      else next_state <= attente; end if;
  end case;
end process;
```

Le testbench de cette architecture est écrit dans le fichier *tb_led_pilot_q7.vhd*.



Les signaux *current_state* et *next_state* sont à « Rouge » au début de la simulation.

La led rouge clignote et la led verte est éteinte.

Lorsque le bouton est pressé, *next_state* passe immédiatement à « Vert ».

Un coup d'horloge après, c'est à *current_state* de passer à « Vert ». En même temps, la led verte s'allume.

Le clignotement est calé sur le signal *led*.

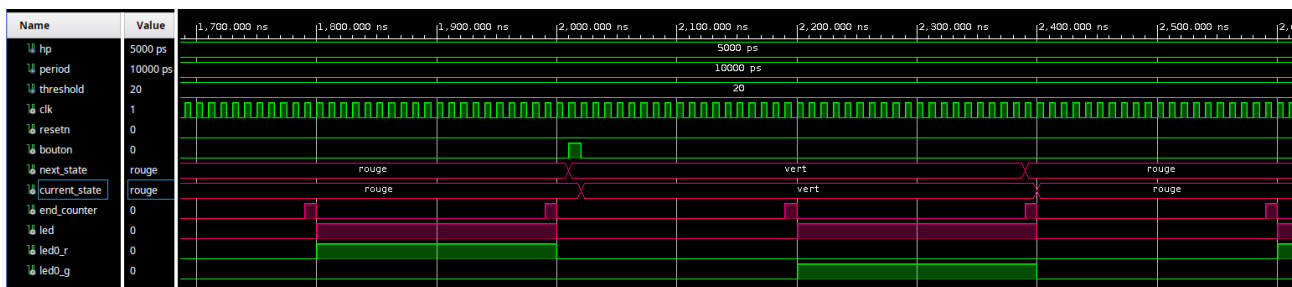
Ici, le bouton a été pressé pendant un niveau haut de *led*, c'est pour cela que la led verte s'allume immédiatement et n'attend pas *end_counter*.

Le bouton reste appuyé, donc lorsque la led verte a clignoté une fois, *next_state* passe à « Attente ». *Current_state* passe ensuite à « Attente » et aucun led ne s'allume.

Lorsque le bouton est relâché, *next_state* puis *current_state* passent à « Rouge ».

Une fois que *current_state* est à « Rouge », la led rouge commence à clignoter.

Il y a simplement un décalage d'un coup d'horloge entre l'appui du bouton et la réaction des leds.



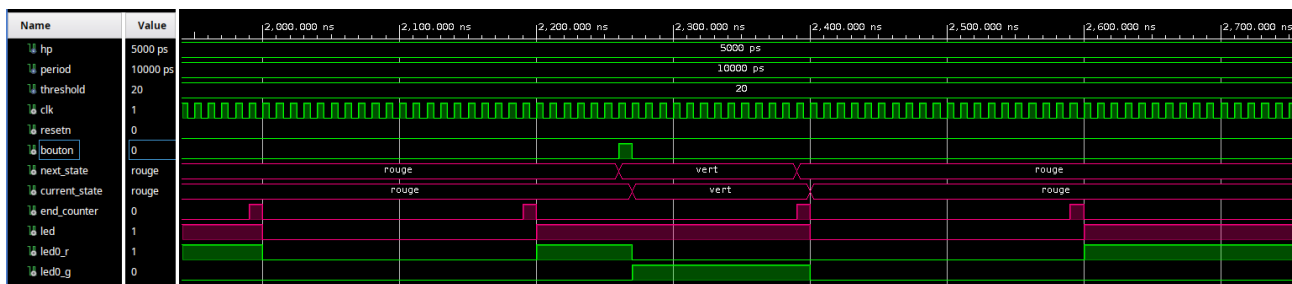
On teste à présent un appui court du bouton.

Lorsque le bouton est pressé, *next_state* puis *current_state* passent à « Vert ».

Le bouton est relâché après un coup d'horloge, malgré cela l'état Vert persiste.

Il faut que la led verte ait le temps de clignoter. Pour cela on attends que *end_counter* et *led* soient à 1. C'est la condition d'une fin de clignotement.

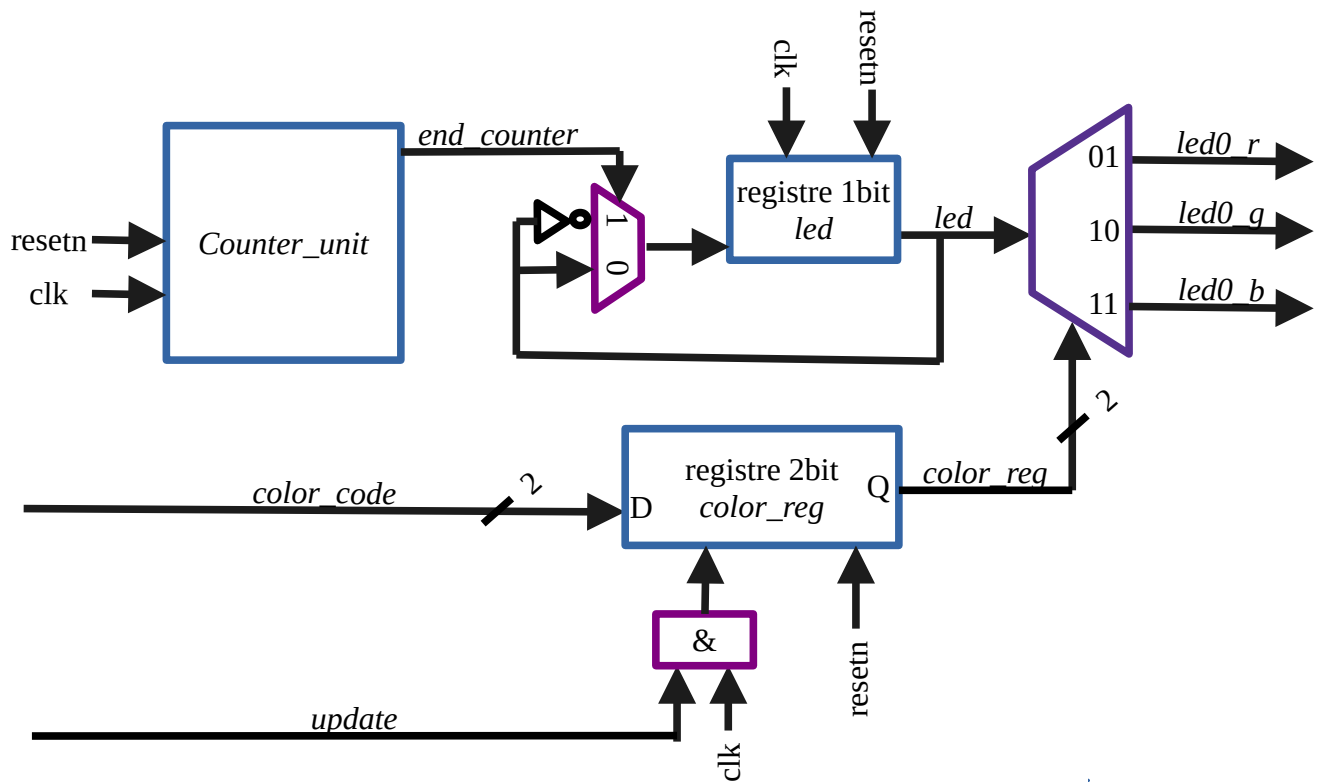
À ce moment, le bouton est déjà relâché donc *next_state* et *current_state* passent directement à l'état « Rouge » sans passer par « Attente ».



À noter que lorsque le bouton est appuyé pendant que la led rouge est allumée, le clignotement de la led verte ne sera pas « complet ». La led verte reste moins longtemps allumée que le temps d'un clignotement.

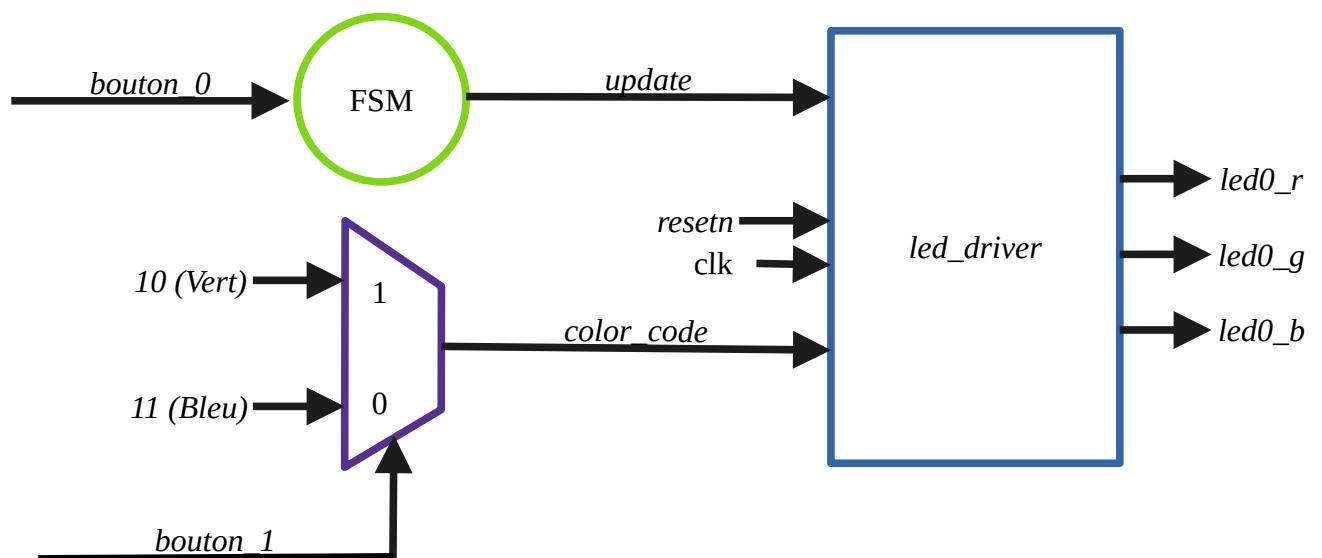
8)

Voici l'architecture du module *led_driver*.

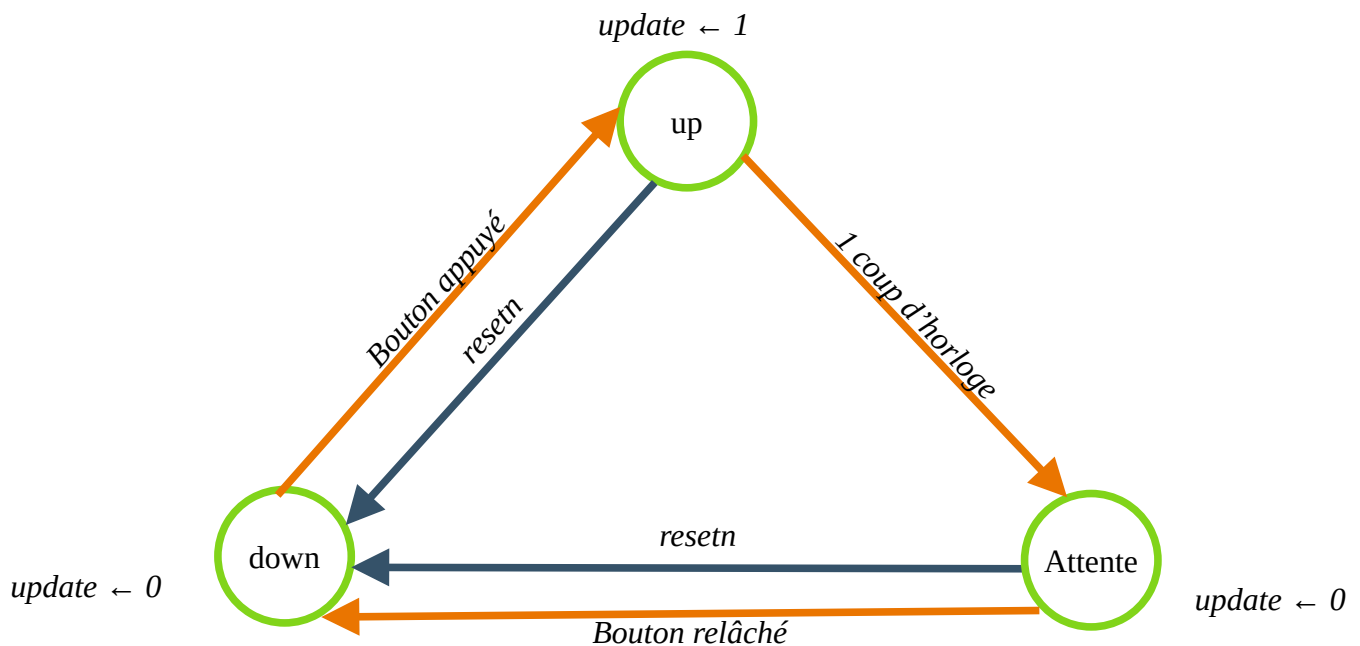


9)

Le module *led_driver* sera instancié dans l'architecture *top_led_driver*.



La machine d'état pour le signal *update* a 3 états. L'état « Attente » permet de remettre *update* à 0 même si le bouton_0 reste enfoncé.

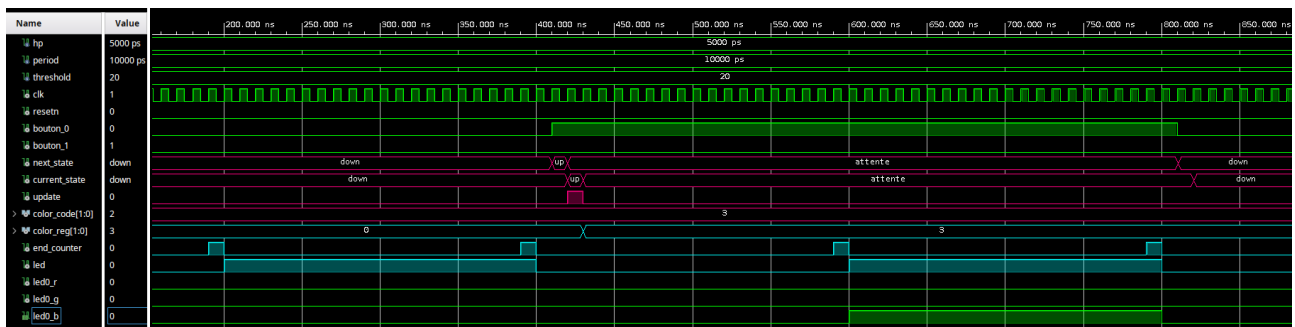


12)

En vert les entrées et sortie de l'architecture.

En rose les signaux internes à *top_led_driver*.

En bleu les signaux internes à *led_driver*.



Les signaux *current_state* et *next_state* commencent à « down »

color_code est à 11₂ (led bleue) car le bouton 1 n'est pas pressé.

color_reg est à 00₂ car c'est sa valeur par défaut après le reset.

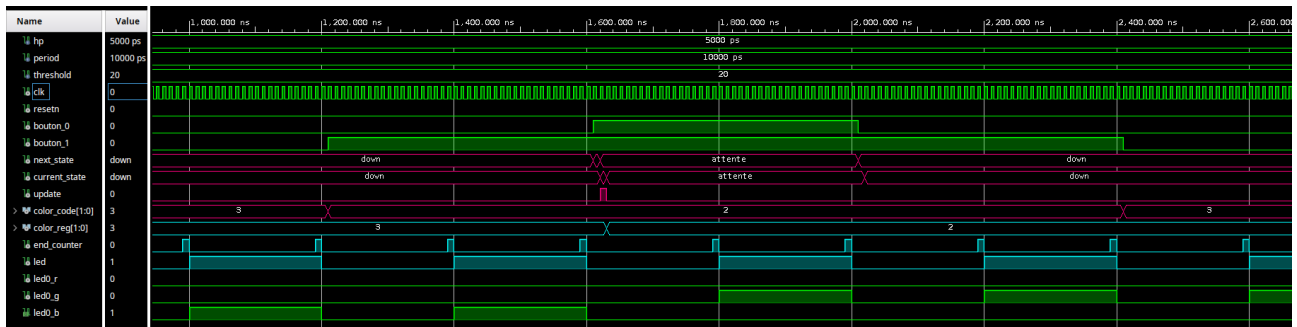
À 410 ns, le bouton 0 est pressé. *next_state* passe alors à l'état « up » et un coup d'horloge après, c'est *current_state* qui passe à « up ».

Au même instant, le signal *update* est lancé. *color_reg* se met alors à jour avec la valeur de *color_code*, soit 11₂ (led bleue)

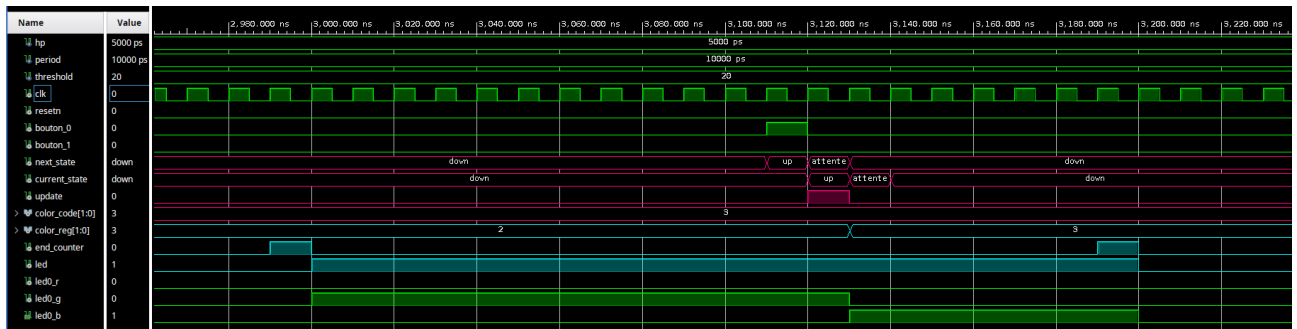
Les signaux *current_state* et *next_state* ne restent qu'un seul coup d'horloge à « up ». Ils passent tout deux à l'état « attente ».

On peut voir que la led bleue clignote car elle s'allument à 600 ns puis s'éteint.

Lorsque le bouton 0 est relâché, *next_state* puis *current state* passent à l'état « down ».



La led bleue continue de clignoter même après avoir lâché le bouton 0.
 Ensuite le bouton 1 est pressé et *color_code* change à 10_2 (led verte).
 La led bleue clignote toujours car le bouton 0 n'est pas encore appuyé.
 Lorsque celui-ci est pressé, *next_state* suivit de *current_state* passent à « up » pour un cycle d'horloge.
 Le signal *update* est envoyé et *color_reg* peut alors se mettre à jour.
 C'est maintenant la led verte qui clignote.
 Elle va encore clignoter quand le bouton 0 est relâché puis le bouton 1.
 Tant que le bouton 0 n'est pas pressé à nouveau, la led verte continuera de clignoter.



Après cela, on teste un appui court sur le bouton 0.
 Pour bien observer le changement d'état, on peut se placer à un moment où la led verte est allumée.
 Le bouton 0 est alors pressé seulement pendant un cycle d'horloge.
 Les signaux *next_state* et *current_state* passent à « up ». Ensuite, ils passent tout de même par l'état « attente » pendant un cycle. Cela ne change pas le comportement des leds.
 Après le signal *update*, *color_reg* est mis à jour et la led passe du vert au bleu.
 Il y a une latence de deux cycles d'horloge entre l'appui du bouton et la réaction des leds.

13)

Avant la synthèse, on connecte les entrées et sorti de l'architecture aux port de la puce.

Pour cela on décommente et renomme les lignes suivantes dans le fichier de contrainte.

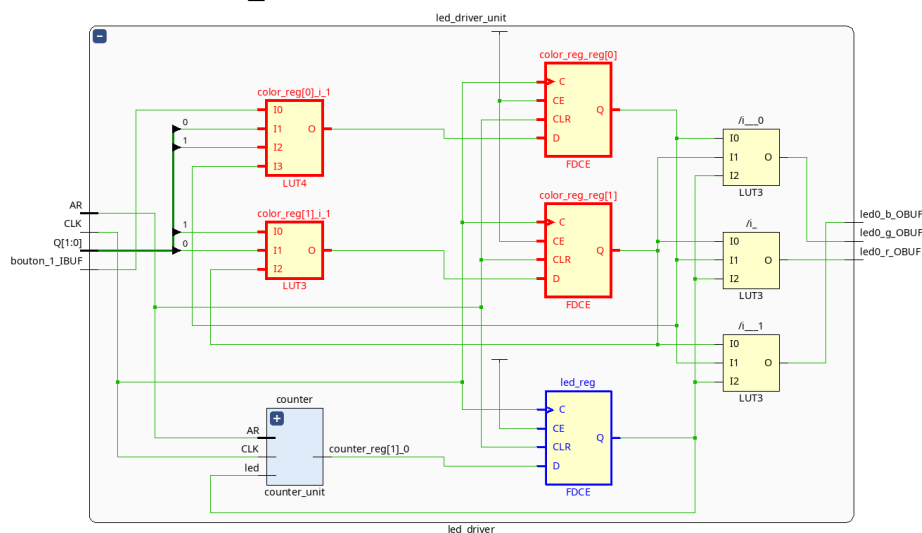
```
# PL System Clock
set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L13P_T2_MRCC_35 Sch=sysclk
create_clock -add -name sys_clk_pin -period 10.00 -waveform { 0 4 } [get_ports { clk }];#set

# RGB LEDs
set_property -dict { PACKAGE_PIN L15 IOSTANDARD LVCMOS33 } [get_ports { led0_b }]; #IO_L22N_T3_AD7N_35 Sch=led0_b
set_property -dict { PACKAGE_PIN G17 IOSTANDARD LVCMOS33 } [get_ports { led0_g }]; #IO_L16P_T2_35 Sch=led0_g
set_property -dict { PACKAGE_PIN N15 IOSTANDARD LVCMOS33 } [get_ports { led0_r }]; #IO_L21P_T3_DQS_AD14P_35 Sch=led0_r
#set_property -dict { PACKAGE_PIN G14 IOSTANDARD LVCMOS33 } [get_ports { led1_b }]; #IO_0_35 Sch=led1_b
#set_property -dict { PACKAGE_PIN L14 IOSTANDARD LVCMOS33 } [get_ports { led1_g }]; #IO_L22P_T3_AD7P_35 Sch=led1_g
#set_property -dict { PACKAGE_PIN M15 IOSTANDARD LVCMOS33 } [get_ports { led1_r }]; #IO_L23N_T3_35 Sch=led1_r

# Buttons
set_property -dict { PACKAGE_PIN D20 IOSTANDARD LVCMOS33 } [get_ports { bouton_0 }]; #IO_L4N_T0_35 Sch=btn[0]
set_property -dict { PACKAGE_PIN D19 IOSTANDARD LVCMOS33 } [get_ports { bouton_1 }]; #IO_L4P_T0_35 Sch=btn[1]

# Reset
set_property -dict { PACKAGE_PIN L19 IOSTANDARD LVCMOS33 } [get_ports { resetn }]; #IO_L9P_T1_DQS_AD3P_35 Sch=user_dio[1]
```

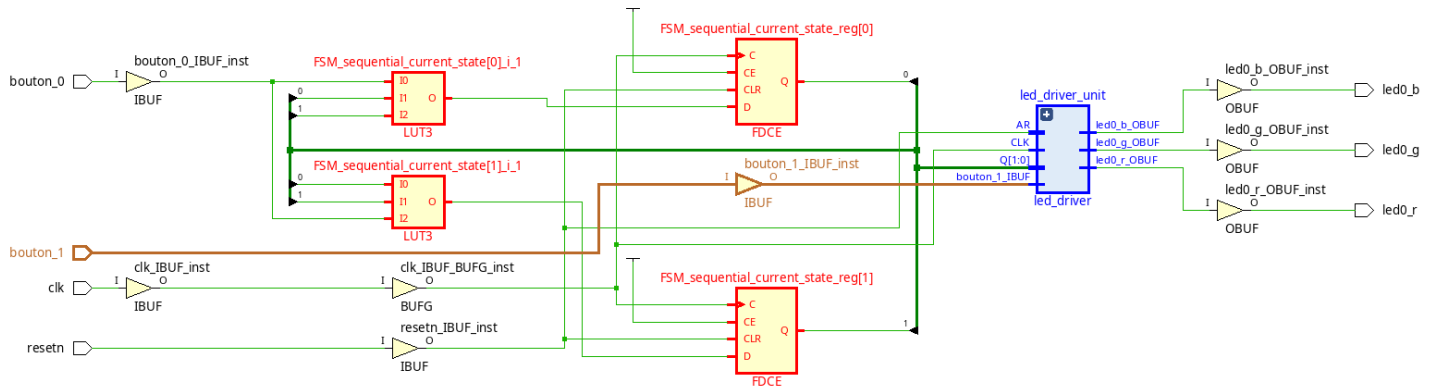
Voici le schematic du module *led_driver*.



En rouge les 2 LUT et les 2 registre de *color_reg*. En bleu le registre de *led_reg* qui prend en entrée le signal *end_counter* provenant du compte.

Le multiplexeur du schémas RTL pour sélectionner la led à faire clignoter, est remplacé par trois LUT ; une par led.

Ci dessous, l'architecture de *top_led_driver*.



En rouge les 2 LUT et les 2 registres de la machine d'état. En bleu le module *led_driver*.
On remarque que l'outil a fait une simplification sur le mutiplexeur entre le bouton 1 et *color_code*.

7. Primitives

Ref Name	Used	Functional Category
FDCE	10	Flop & Latch
LUT3	6	LUT
IBUF	4	IO
OBUF	3	IO
LUT5	2	LUT
LUT4	2	LUT
LUT6	1	LUT
LUT2	1	LUT
LUT1	1	LUT
BUFG	1	Clock

Le rapport d'utilisation montre que 10 registres et 13 LUTs sont utilisés.
6 LUTs et 5 registres sont occupés par le compteur.
5 LUTs et 3 registre pour *led_driver*.
2 LUTs et 2 registre pour *top_led_driver*.

Les schematics et l'utilisation matériel sont cohérent avec les schémas RTL établis avant le codage VHDL.

14)

Après implémentation, le rapport de timing donne des TNS et THS nuls.

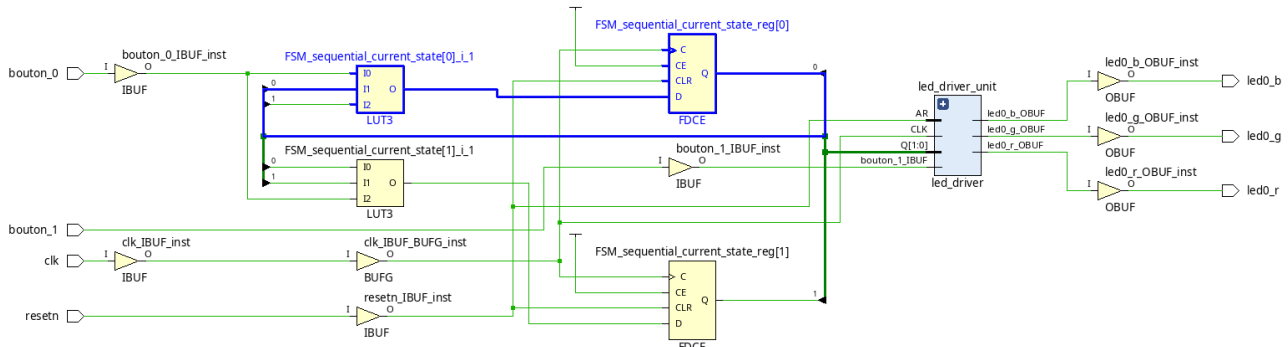
Le chemin critique a un slack de 8,328ns. L'horloge étant réglée sur 100MHz, elle a une période de 10ns.

Le rapport montre que le chemin critique a un délais total de 1,714ns.

Or $1,714 + 8,328 = 10,042$ ns.

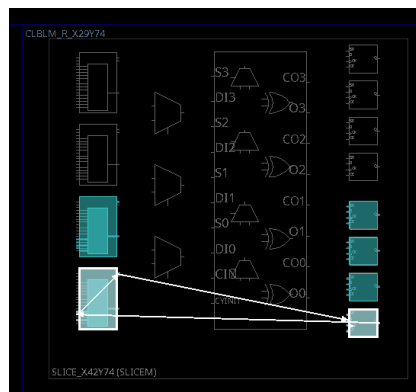
Je ne sais pas à quoi correspondent ces 42µs en plus par rapport à la période de l'horloge.

Vivado fait peut être une approximation dans ses calculs qui conduisent à ces 42µs.



Le chemin critique reboucle sur lui-même. Il part et arrive à un registre de la machine d'état.

Celui-ci est en fait contenu dans une seule slice. Il fait une boucle entre un registre et une LUT de la même slice.



15)

Le bitstream est généré correctement mais je n'ai pas encore la carte pour vérifier l'architecture.